

产生重要影响,因此用户参与是数据库设计不可或缺的一部分。在需求分析阶段,任何调查研究没有用户的积极参与都是难以进行的。数据库设计员应和用户取得共同的语言,帮助不熟悉计算机的用户建立数据库环境下的共同概念,并对设计工作的最后结果承担共同的责任。

7.3 概念结构设计

概念结构设计就是将需求分析得到的用户需求抽象为信息结构(即概念模型)的过程。它是整个数据库设计的关键。本节介绍概念模型的特点,重点讲解用 E-R 模型来表示概念模型的方法。

7.3.1 概念模型

在需求分析阶段得到的应用需求应该首先抽象为信息世界的结构,然后才能更好、更准确地用某一数据库管理系统实现这些需求。

概念模型的主要特点是:

① 能真实、充分地反映现实世界,包括事物和事物之间的联系,能满足用户对数据的处理要求,是现实世界的一个真实模型。

② 易于理解,可以用它和不熟悉计算机的用户交换意见。用户的积极参与是数据库设计成功的关键。

③ 易于更改,当应用环境和应用要求改变时容易对概念模型进行修改和扩充。

④ 易于向关系模型、网状模型、层次模型等各种数据模型进行转换。

概念模型是各种数据模型的共同基础,它比数据模型更独立于机器、更抽象,从而更加稳定。描述概念模型的有效工具是 E-R 模型。

7.3.2 E-R 模型

P. P. S. Chen 提出的 E-R 模型是用 E-R 图来描述现实世界的概念模型。第 1 章 1.2.2 小节初步介绍了 E-R 模型涉及的主要概念,包括实体、属性、实体之间的联系等,指出实体应该区分实体集和实体型。下面对实体之间的联系做进一步介绍,然后讲解 E-R 图。

1. 实体之间的联系

在现实世界中,事物内部以及事物之间是有联系的。实体内部的联系通常是指组成实体的各属性之间的联系,实体之间的联系通常是指不同实体型的实体集之间的联系。

(1) 两个实体型之间的联系

两个实体型之间的联系可以分为以下三种:

① **一对一联系(1:1)**。如果对于实体型 A 中的每一个实体,实体型 B 中至多有一个(也可以没有)实体与之联系,反之亦然,则称实体型 A 与实体型 B 具有一对一联系,记为 1:1。例

如,学校的某个学院只有一位教师任职院长,而一位教师只在一个学院中任职院长,则教师与学院之间具有一对一联系,联系名为“担任院长”。

② **一对多联系(1:n)**。如果对于实体型 A 中的每一个实体,实体型 B 中有 n 个实体 ($n \geq 0$) 与之联系;反之,对于实体型 B 中的每一个实体,实体型 A 中至多只有一个实体与之联系,则称实体型 A 与实体型 B 有一对多联系,记为 $1:n$ 。例如,一个学院中设置了若干个系,而每个系只能归属于一个学院,则学院与系之间具有一对多联系。又如,一门课程可以开设多个教学班,而一个教学班只能归属于一门课程,则课程和教学班之间具有一对多联系。

③ **多对多联系(m:n)**。如果对于实体型 A 中的每一个实体,实体型 B 中有 n 个实体 ($n \geq 0$) 与之联系;反之,对于实体型 B 中的每一个实体,实体型 A 中也有 m 个实体 ($m \geq 0$) 与之联系,则称实体型 A 与实体型 B 具有多对多联系,记为 $m:n$ 。例如,一门课程同时有若干学生选修,而一个学生可以同时选修多门课程,则课程与学生之间具有多对多联系。

可以用图形来表示两个实体型之间的这三类联系,如图 7.6 所示。

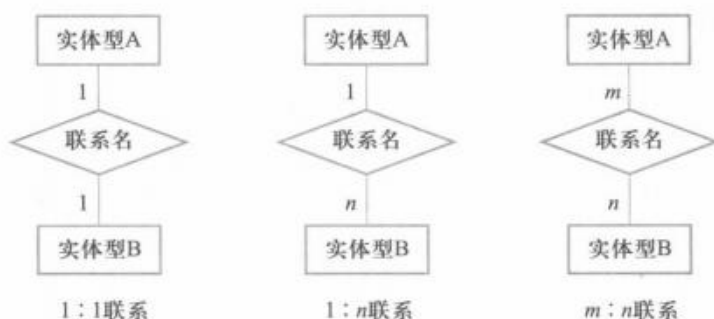


图 7.6 两个实体型之间的三类联系

(2) 两个以上实体型之间的联系

一般地,两个以上的实体型之间也存在一对一、一对多和多对多联系。

例如,对于学生、课程、教师三类实体型,如果每个学生可以对其选修的多门课程中每一个授课教师单独进行课程评价,每个教师也可以针对其讲授的多门课程中每一个学生的课程评价进行意见反馈,则学生、课程、教师之间的课程评价联系是多对多的,如图 7.7(a) 所示。

又如,对于供应商、项目、零件三类实体型,一个供应商可以供给多个项目多种零件,而每个项目可以使用多个供应商供应的零件,每种零件可由不同的供应商供给,由此看出供应商、项目、零件三者之间是多对多的供应联系,如图 7.7(b) 所示。

(3) 单个实体型内的联系

同一个实体型内的各实体之间也可以存在一对一、一对多和多对多的联系。例如,课程实体型内部具有“先修”的联系,在第 2 章中列出的“先修课”Cpno 是直接先修课,即假设一门课只能列出一门直接先修课,该门课可以是多门课程的直接先修课,所以课程内部的“先修”是

一对多的联系,如图 7.8(a)所示。这是做了简化假设的。

实际上一门课程也可以有多门直接先修课,某一门课程也可以作为多门课程的先修课,这时课程内部的“先修”是多对多的联系,如图 7.8(b)所示。

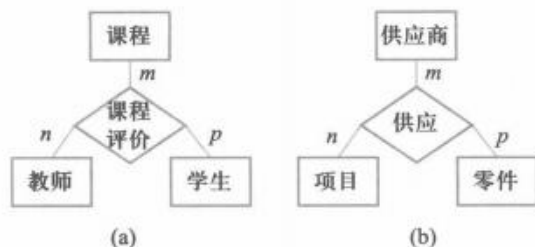


图 7.7 三个实体型之间的联系示例

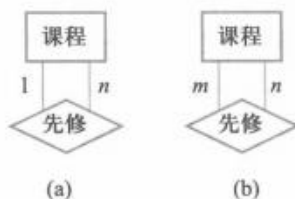


图 7.8 单个实体型内的联系示例

注意: 实体型之间可能存在多种联系。例如,系与教师之间存在一对多的工作联系,也可以存在一对一的系主任联系。

2. E-R 图

E-R 图提供了表示实体型、属性和联系的方法。

① 实体型用矩形表示,矩形框内写明实体名。

② 属性用椭圆形表示,并用无向边将其与相应的实体型连接起来。例如,学生实体型具有学号、姓名、性别、出生日期属性,用 E-R 图表示如图 7.9 所示。

③ 联系用菱形表示,菱形框内写明联系名,并用无向边分别与有关实体型连接起来,同时,在无向边旁标注联系的类型(如 1:1、1:n 或 m:n 等)。

需要注意的是,如果一个联系具有属性,则这些属性也要用无向边与该联系连接起来。

例如在图 7.7(b)中,如果用“供应量”来描述联系“供应”的属性,表示某供应商供应了多少数量的零件给某个项目,那么这三个实体型及其之间联系的 E-R 图表示应如图 7.10 所示。



图 7.9 实体型及属性 E-R 图表示示例

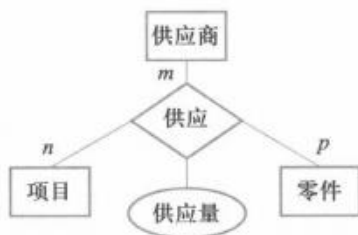


图 7.10 实体型及联系 E-R 图表示示例

3. 一个实例

下面用 E-R 图来表示“学生学籍管理”子系统的概念模型。

设“学生学籍管理”子系统涉及的 5 类实体型及属性如下。

① 学院: 学院编号、学院名、建院时间。

- 这些实体型之间的联系如下:

⑤ 一个专业同时有若干个学生选择, 一个学生可以选择一个专业作为主修, 另外若干个专业作为辅修。因此, “专业”和“学生”实体型之间具有多对多联系。

注意：第3章中一个学生只能选择一个专业。按照高校目前的实际情况，一个学生可以选择多个专业，我们的概念模型也做了相应的扩展和调整。

“学生学籍管理”子系统的分 E-R 图如图 7.11 所示。

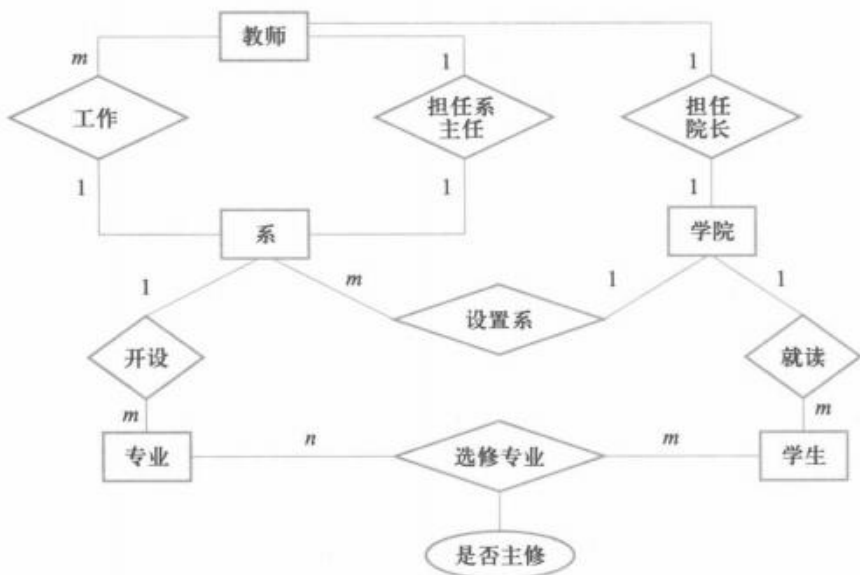


图 7.11 “学生学籍管理”子系统的分 E-R 图

7.3.3 扩展的 E-R 模型

E-R 模型是抽象和描述现实世界的有力工具。用 E-R 图表示的概念模型独立于具体的数据库管理系统所支持的数据模型,是各种数据模型的共同基础,因而 E-R 模型比数据模型更一般、更抽象,也更接近现实世界。

E-R 模型得到了广泛的应用,人们在基本 E-R 模型的基础上进行了某些方面的扩展,使其表达能力更强。

1. ISA 联系

用 E-R 模型构建一个项目的模型时,经常会遇到某些实体型是某个实体型的子类型。例如,研究生和本科生两类实体型是学生实体型的子类型,学生实体型是父类型。这种父类-子类联系称为 ISA 联系,表示“is a”语义。例如在图 7.12 中,研究生 is a 学生,本科生 is a 学生。ISA 联系用三角形来表示。

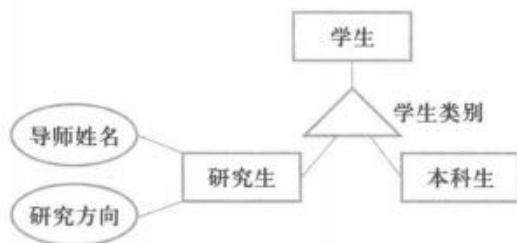


图 7.12 ISA 联系示例

ISA 联系的一个重要性质是子类继承了父类的所有属性,当然子类也可以有自己的属性。例如,图 7.12 中“本科生”和“研究生”是“学生”实体型的子类型,它们具有“学生”实体型的全部属性,此外,“研究生”实体型还有“导师姓名”和“研究方向”两个自己的属性。

ISA 联系描述了对一个实体型中实体的一种分类方法,下面对分类方法做进一步说明。

(1) 分类属性

根据分类属性的值把父实体型中的实体分派到子实体型中。例如,图 7.12 中在 ISA 联系符号三角形的右边加了一个分类属性“学生类别”,它说明一个学生是研究生还是本科生由“学生类别”这个分类属性的值决定。

(2) 不相交约束与可重叠约束

不相交约束描述父类中的一个实体不能同时属于多个子类中的实体集,即一个父类中的实体最多属于一个子类实体集,用 ISA 联系三角形符号内加一个叉号“x”来表示。例如,图 7.13 表明一个学生不能既是本科生又是研究生。如果父类中的一个实体能同时属于多个子类中的实体集,则称为可重叠约束,子类符号中没有叉号即表示是可重叠的。

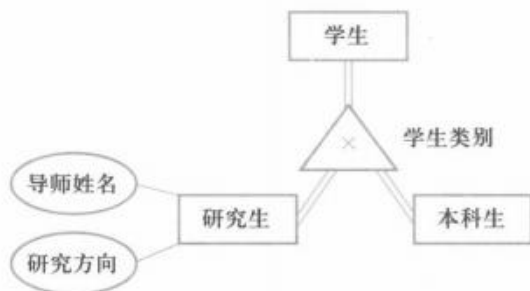


图 7.13 不相交约束示例

(3) 完备性约束

完备性约束描述父类中的一个实体是否必须是某一个子类中的实体，如果是，则称为完全特化(full specialization)，否则称为部分特化(partial specialization)。完全特化用父类到子类的双线连接来表示，单线连接则表示部分特化。假设学生只有两类，要么是本科生，要么是研究生，二者必居其一，这就是完全特化的例子，如图 7.13 所示。

2. 基数约束

基数约束是对实体型之间一对一、一对多和多对多联系的细化。参与联系的每个实体型用基数约束来说明其中的任何一个实体可以在联系中出现的最少次数和最多次数。

约束用一个数对 $\min..max$ 表示， $0 \leq \min \leq max$ 。例如， $0..1$ 、 $1..3$ 、 $1..*$ （* 代表无穷大）。 $\min=1$ 的约束称为强制参与约束，即被施加基数约束的实体型中的每个实体都要参与联系。 $\min=0$ 的约束称为非强制参与约束，即被施加基数约束的实体型中的实体可以出现在联系中，也可以不出现在联系中。本书中，两个实体型之间联系的基数约束标注在远离施加约束的实体型并靠近参与联系的另外一个实体型的位置。例如，图 7.14(a)“学生”和“学生证”实体型的联系中，一个学生必须拥有一本学生证，一本学生证只能属于一个学生，因此都是 $1..1$ 。

在图 7.14(b)中“学生”和“课程”实体型是多对多的联系。假设“学生”实体型的基数约束是 $20..30$ ，表示每个学生必须参与选修课程，并且选修的课程数在 $[20, 30]$ 范围内；课程的一个合理的基数约束是 $0..*$ ，即一门课程一般会被很多学生选修，但是有的课程可能还没有任何一个学生选修，如新开课。

在图 7.14(c)“学院”和“系”实体型的一对多联系中，一个系必须隶属于一个学院，且只能隶属于一个学院，因此“系”的约束为 $1..1$ ，标在参与联系的“学院”实体型附近。一个学院可以设置多个系，考虑到新成立的学院，可能尚未设立系，则学院的基数约束为 $0..*$ 。

采用基数约束的表示方式，一是可以方便地读出约束的类型（一对一、一对多、多对多），如学院和系是一对多的联系；二是一些 E-R 辅助绘图工具也是采用这样的表现形式。

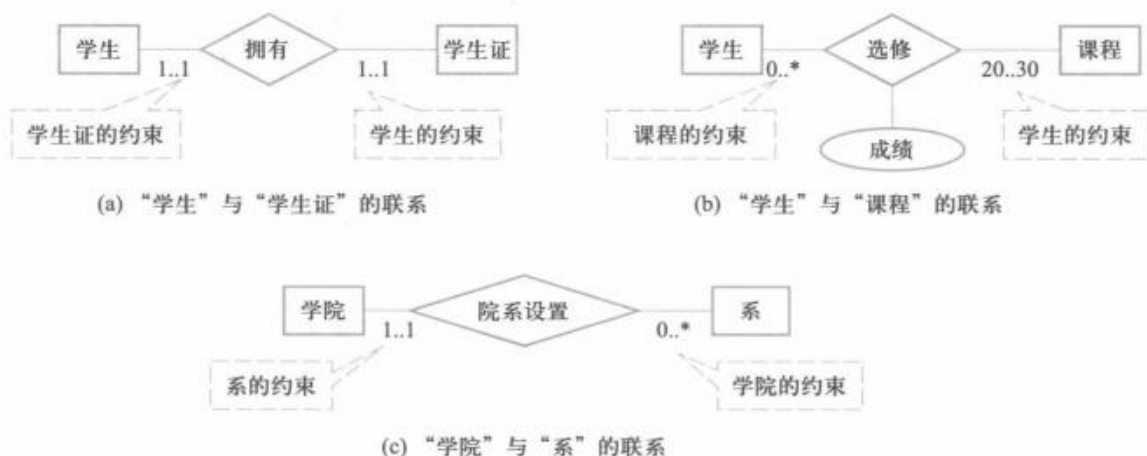


图 7.14 基数约束示例

3. Part-of 联系

Part-of 联系即部分联系，它表明某个实体型是另外一个实体型的一部分。例如汽车和轮子两个实体型，轮子实体型是汽车实体型的一部分，即 Part-of 汽车实体型。

Part-of 联系可以分为两种情况，一种是整体实体如果被破坏，部分实体仍然可以独立存在，称为非独占的 **Part-of** 联系，简称非独占联系。例如，汽车实体型和轮子实体型之间的联系，一辆汽车车体被损毁了，但是轮子还存在，可以拆下来独立存在，也可以再安装到其他汽车上。非独占的 Part-of 联系可以通过基数约束来表达。在图 7.15 中，“汽车”实体型的基数约束是 4..4，即一辆汽车要有 4 个轮子。“轮子”实体型的基数约束是 0..1，这样的约束表示非强制参与联系。它表示一个轮子可以安装到一辆汽车上，也可以没有被安装到任何车辆上而独立存在，即一个轮子可以参与一个联系，也可以不参与。因此，在 E-R 图中用非强制参与联系表示非独占的 Part-of 联系。

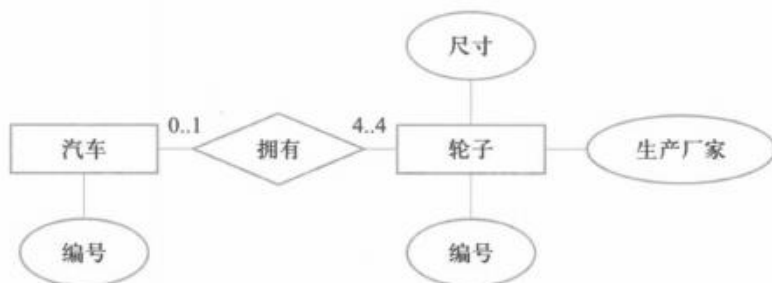


图 7.15 用非强制参与联系表示非独占的 Part-of 联系

与非独占联系相反，还有一种是独占的 **Part-of** 联系，简称独占联系。即整体实体如果被破坏，部分实体不能存在，在 E-R 图中用弱实体类型和识别联系来表示独占联系。如果一个

实体型的存在依赖于其他实体型的存在,则这个实体型叫作弱实体型,否则叫作强实体型。前面介绍的绝大多数实体型都是强实体型。一般来讲,如果不能从一个实体型的属性中找出可以作为码的属性,则这个实体型就是弱实体型。在 E-R 图中用双矩形表示弱实体型,用双菱形表示识别联系。

例如,如图 7.16 所示,在大学的课程设置中,一门课程可以在每个学年甚至是每个学期开设多个教学班,则“教学班”就是一个弱实体型,它只有“教学班号”“人数上限”“开课学期”三个属性。该门课程的第一个教学班号为 1,第二个教学班号为 2,依此类推。这些属性的任何组合都不能作为教学班的码。教学班的存在必须依赖于课程,没有课程自然就没有教学班。

再看一个例子,教室和教学楼之间的联系。如图 7.17 所示,每座教学楼都有唯一的编号或者名称,每间教室都有一个编号,如果教室号不包含楼号,则教室号不能作为码,因为不同的教学楼中可能有编号相同的教室,所以“教室”是一个弱实体。例如,教一楼和教二楼都有“101 号”教室,但教室号没有包含楼号,所以该教室号不能作为码。

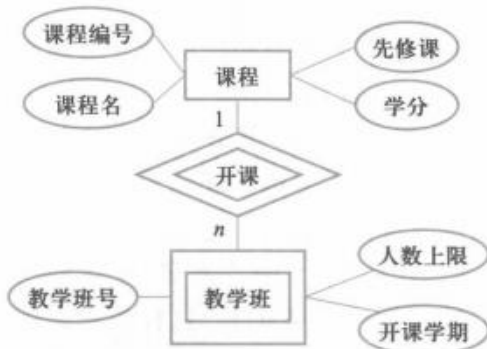


图 7.16 弱实体型和识别联系

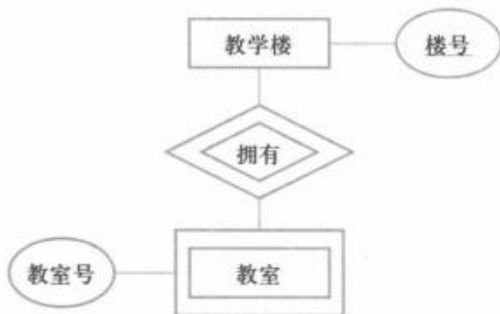


图 7.17 教室是一个弱实体

注意：由于 E-R 图的图形元素并没有标准化,不同教材和构建 E-R 图的不同工具软件之间都会有一些差异。

*7.3.4 用 UML 中的类图表示 E-R 图

表示 E-R 图的方法有若干种,使用统一建模语言 UML 即为其中之一。

UML 是对象管理组 (Object Management Group, OMG) 的一个标准。它不是专门针对数据建模的,而是为软件开发的所有阶段提供模型化和可视化支持的一种规范语言。从需求规格描述到系统完成后的测试和维护都可以用到 UML。UML 可用于数据建模、业务建模、对象建模、组件建模等,它提供了多种类型的模型描述图,借助这些图可以使计算机应用系统开发中的应用程序更易理解。关于 UML 的概念、内容和使用方法等可以专门开设一门课程来讲解,已经超出本书范围,这里仅简单介绍如何用 UML 中的类图来建立概念模型(即 E-R 图)。

UML 中的类(class)大致对应 E-R 图中的实体。由于 UML 中的类具有面向对象的特征,它不仅描述对象的属性,还包含对象的方法(**method**)。方法是面向对象技术中的重要概念,在对象关系数据库中支持方法,但 E-R 模型和关系模型都不提供方法,因此本书在用 UML 表示 E-R 图时省略了对象方法的说明。

1. 实体型

在 UML 中用类表示实体型。矩形框中实体名放在上部,下面列出属性名。

2. 实体的主码

在类图中的某一个或若干个属性后面加“PK”(primary key)表示实体的主码属性。

3. 联系

用类图之间的“关联”来表示联系。早期的 UML 只能表示二元关联,关联的两个类用无向边相连,在连线上写关联的名称。例如,学生、课程、学生与课程之间的联系,以及基数约束的 E-R 图用 UML 的类图表示如图 7.18 所示。现在 UML 也扩展了非二元关联,并用菱形框表示关联,框内写联系名,用无向边分别与关联的类连接起来。



图 7.18 用 UML 的类图表示 E-R 图示例

4. 基数约束

UML 中关联类之间基数约束的概念及表示和 E-R 图中的基数约束类似。用一个数对 min..max 表示类中的任何一个对象可以在关联中出现的最少次数和最多次数。例如, 0..1、1..3、1..*。基数约束的标注方法和 7.3.3 小节中介绍的一样,在图 7.18 中,“学生”和“课程”的基数约束标注“20..30”表示每个学生必须选修 20~30 门课程, 0..* 表示一门课程一般会被很多学生选修,但也可能没有学生选修。

5. UML 中的子类

面向对象技术支持超类-子类概念,子类可以继承超类的属性,也可以有自己的属性。这些概念和 E-R 图的父类-子类联系、ISA 联系是一致的。因此很容易用 UML 表示 E-R 图的父类-子类联系。

注意: 如果计算机应用系统的设计和开发的全过程使用了 UML 规范,开发人员自然可以采用 UML 对数据建模。否则,建议数据库设计采用 E-R 模型来表示概念模型。

7.3.5 用 E-R 图进行概念结构设计

概念结构设计的第一步就是对需求分析阶段收集到的数据进行分类、组织，确定实体、实体的属性、实体之间的联系类型，形成 E-R 图。其中，如何确定实体和属性这个看似简单的问题常常会困扰设计人员，因为实体与属性之间并没有形式上可以截然划分的界限。本节介绍在设计 E-R 图的过程中如何确定实体与属性，以及在集成 E-R 图时如何解决冲突等关键技术。

1. 实体与属性的划分原则

事实上，在现实世界中具体的应用环境常常已对实体和属性做了自然的大体划分。在数据字典中，数据结构、数据流和数据存储都是若干有意义的属性聚合，这就已经体现了这种划分。可以先从这些内容出发定义 E-R 图，然后再进行必要的调整。在调整中遵循的一条原则是：为了简化 E-R 图的处置，现实世界的事物能作为属性的尽量作为属性对待。

那么，符合什么条件的事物可以作为属性对待呢？可以给出两条准则：

① 属性不能再具有需要描述的性质。属性必须是不可分的数据项，不能包含其他属性。

② 属性不能与其他实体具有联系。E-R 图中所表示的联系是实体之间的联系。

凡满足上述两条准则的事物，一般均可作为属性对待。

例如，“教师”是一个实体型，“职工号”“姓名”“出生日期”是“教师”的属性，“职称”如果没有与工资、岗位津贴、福利等关联，换句话说，没有需要进一步描述的特性，则根据准则①可以作为“教师”实体的属性；但如果需要描述和处理不同职称的工资、岗位津贴和不同的附加福利等信息，则“职称”作为一个实体对待就更恰当，不作为属性对待，如图 7.19 所示。

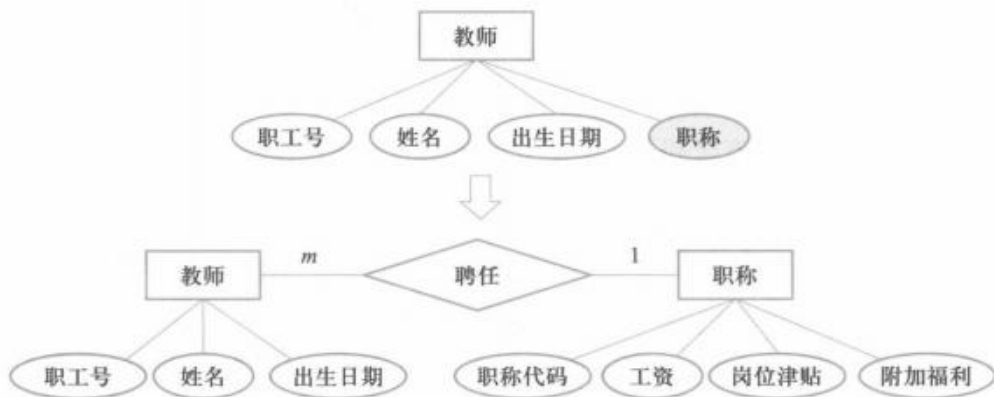


图 7.19 “职称”作为一类实体

又如，一个学生如果只能选择一个专业，且专业没有和专业类别、专业年限等关联，则

可以把“主修专业”作为“学生”实体的属性(如第3章中“主修专业”语义);但如果一个学生可以选择多个专业(例如其中有一个专业为主修专业,其他专业为辅修专业),则需要把“专业”单独作为一类实体对待,并建立“学生”与“专业”的多对多“专业选择”联系,如图7.20所示。

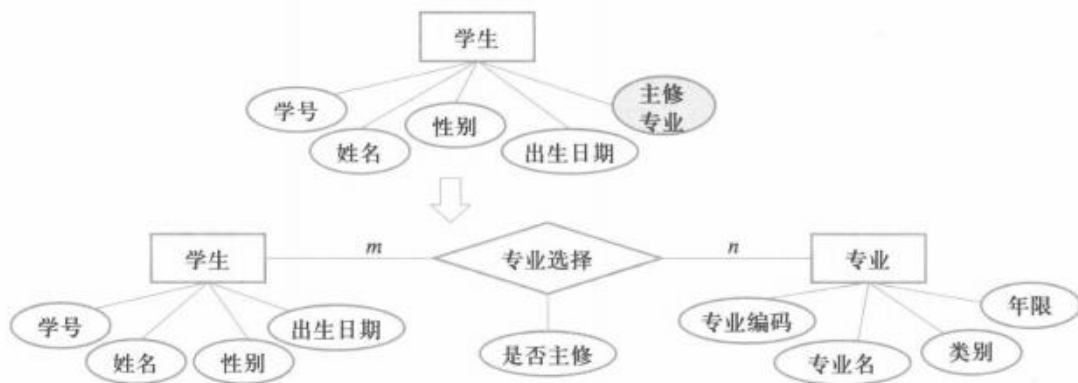


图 7.20 “专业”作为一类实体

再如,一门课程如果只存在一门直接的先修课,则可以把先修课的课程号作为“课程”实体的属性(如第3章中“课程”的语义),属性名为“先修课”;但如果一门课程存在多门直接的先修课,则需要把“先修课”作为单独的一类实体。由于“先修课”实体型也是“课程”实体型,因此“先修课”与“课程”之间实际上是单个实体型内的多对多联系,构建的结果如图7.21所示。



图 7.21 “先修课”作为一类实体(与“课程”实体型相同)

另外,如果一门课程只设一个教学班,则可以把教学班号作为“选课”联系的属性,属性名为“教学班”,如图7.22(a)所示。但在实际的教学活动中,一门课程通常可以开设多个教学班,则应把“教学班”作为一类实体,如图7.22(b)所示。注:为了方便叙述,图7.22(b)省略了“学生”“课程”和“教学班”实体型的属性。

[例 7.1]“教师教学管理”子系统的分E-R图的设计。

某高校教务处开发“高校本科教务管理”信息系统,经过可行性分析,详细调查确定了该系统由“学生学籍管理”“教师教学管理”“学生选课管理”等子系统组成。每个子系统均成立了开发小组,设计该子系统的分E-R图。图7.22已给出了“学生选课管理”子系统的分E-R图,下面设计“教师教学管理”子系统的分E-R图。

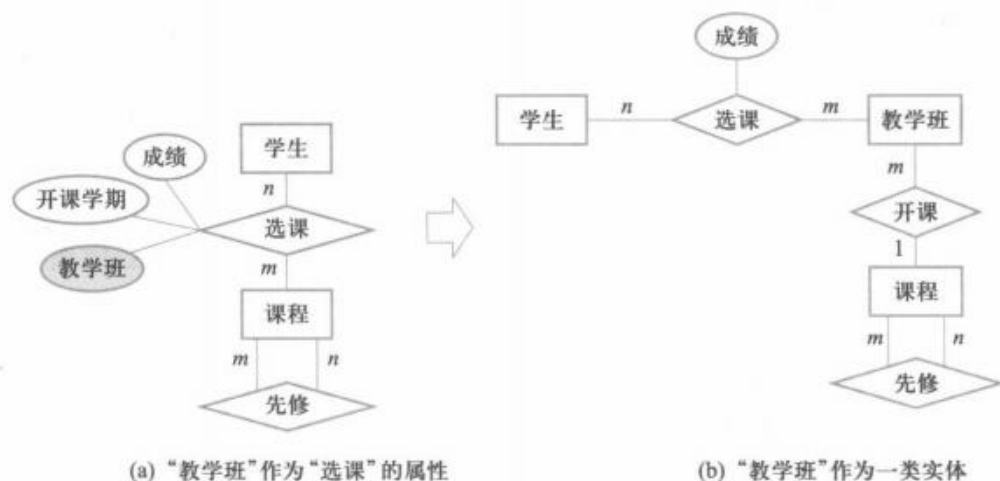


图 7.22 “学生选课管理”子系统的分 E-R 图

“教师教学管理”子系统开发小组的成员经过需求调查、处理流程分析和数据收集，明确了该子系统的主要功能如下：

- ① 为每个教学班安排授课教师。
- ② 为每个教学班排课，设置教室、上课时间。
- ③ 针对学生对教学班的课程评价，授课教师需要对学生的意见进行反馈。

该子系统涉及的实体及属性如下：

- ① 教师：职工号、姓名、出生日期、职称（这里为方便讨论，将“职称”作为“教师”的属性）。
- ② 教学班：教学班号、人数上限、开课学期。
- ③ 学生：学号、姓名、性别、出生日期。
- ④ 教室：教室号（教室号已包含了楼号，如教室号“003101”表示教3楼101教室）、教学楼号、联系人、联系方式。

⑤ 时间片：时间片编码（如“10102”表示星期一第1节课开始，第2节课下课结束）、星期几、开始时间、截止时间。“时间片”实体也可以作为“教学班”与“教师”的“排课”联系属性。考虑到同一个教学班在一周内可以安排在相同的教室、但在不同的时间段讲授多次，这里将“时间片”单独作为实体类。

这些实体之间的联系如下：

① 一个教学班可以安排多个教师来讲授，一个教师也可以讲授多个教学班，因此“教师”与“教学班”实体之间具有多对多的“讲授”联系，联系的属性包括“是否为主讲教师”（一个教学班设置一个主讲教师），如图 7.23 所示。

② 学生可以针对其选择的教学班中的任一位老师进行课程评价，授课教师也需要对其讲授的所有教学班中每一位学生提出的评价进行一一反馈。因此，“教师”“学生”“教学班”三个

实体之间就“课程评价”关联存在多对多联系,联系属性包括“评价内容”“评价类型”(正面/负面)和“教师反馈”,如图 7.24 所示(图中省略了各实体型的属性)。



图 7.23 “教师”与“教学班”实体之间的“讲授”联系

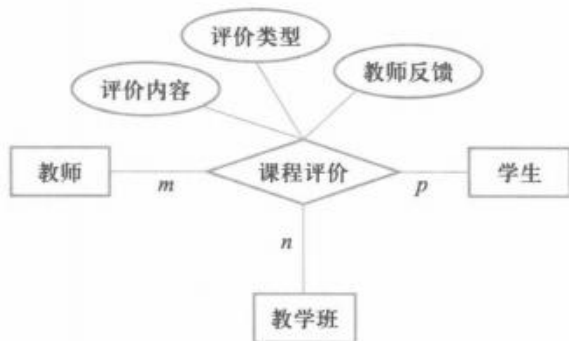


图 7.24 “教师”“教学班”“学生”三个实体之间的“课程评价”联系

③“教室”与“教学班”和“时间片”实体之间的“排课”联系属于一对多联系,即确定了某个教学班、某个时间片,则教室可以唯一确定(一个教室在同一时间段只能安排一门课程),如图 7.25 所示。

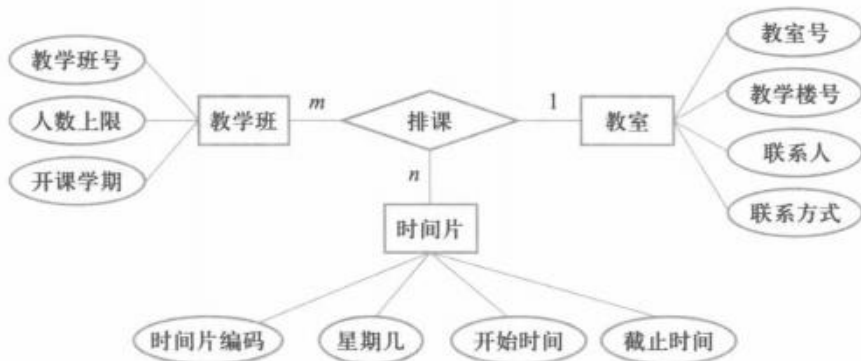


图 7.25 “教室”“教学班”“时间片”三个实体之间的“排课”联系

“教师教学管理”子系统的分 E-R 图如图 7.26 所示。

2. E-R 图的集成

在开发一个大型信息系统时最常采用的策略是自顶向下地进行需求分析,然后再自底向上地设计概念结构。即首先设计各子系统的分 E-R 图,然后将它们集成起来,得到全局 E-R 图。E-R 图的集成一般需要分两步走,如图 7.27 所示,第一步是合并,即解决各分 E-R 图之间的冲突,将分 E-R 图合并起来生成初步 E-R 图。第二步是修改与重构,即消除不必要的冗余,生成基本 E-R 图。

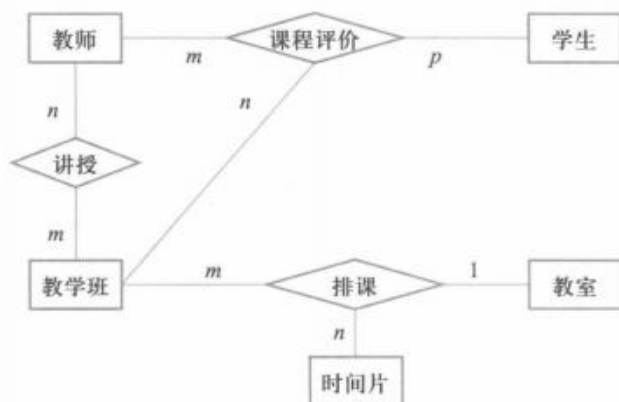


图 7.26 “教师教学管理”子系统的分 E-R 图

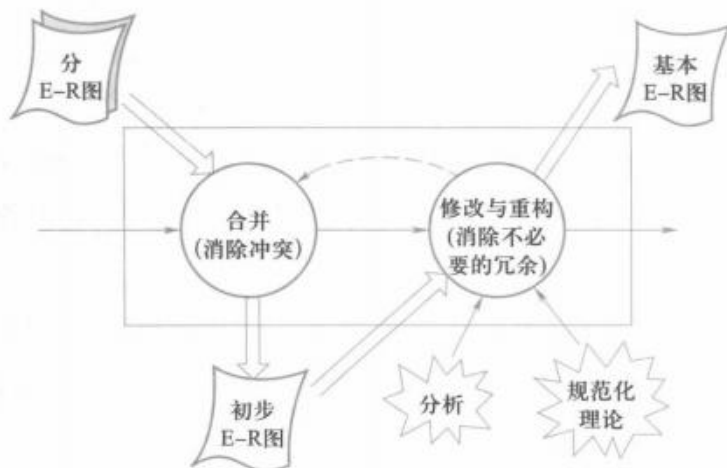


图 7.27 E-R 图的集成示意图

下面分别介绍这两个步骤。

(1) 合并 E-R 图，生成初步 E-R 图

各个局部应用所面向的问题不同，且通常是由不同的设计人员进行局部视图设计，这就导致各个子系统的 E-R 图之间必定会存在许多不一致的地方，称之为“冲突”。因此，合并这些 E-R 图时并不能简单地将各个 E-R 图画到一起，而是必须着力消除各个 E-R 图中的不一致，以形成一个能为全系统中所有用户共同理解和接受的统一的概念模型。合理地消除各 E-R 图的冲突是合并 E-R 图的主要工作与关键所在。

各子系统的 E-R 图之间的冲突主要有三类：属性冲突、命名冲突和结构冲突。其中，属性冲突主要包含以下两类冲突：

① 属性域冲突，即属性值的类型、取值范围或取值集合不同。例如教学班号，有的部门把它定义为整数，有的部门则把它定义为字符型，不同部门对它的编码也不同。又如年龄，某些部门以出生日期形式表示职工的年龄，而另一些部门则用整数表示职工的年龄。

② 属性取值单位冲突。例如,出生日期有的精确到年,有的精确到月,有的精确到日。

属性冲突理论上好解决,但实际上需要各部门讨论协商,解决起来并非易事。

命名冲突主要包含以下两类冲突:

① 同名异义,即不同意义的对象在不同的局部应用中具有相同的名字。

② 异名同义(一义多名),即同一意义的对象在不同的局部应用中具有不同的名字。例如对于科研项目,财务科称其为“项目”,科研处称其为“课题”,而生产管理处则称其为“工程”。

命名冲突可能发生在实体、联系一级上,也可能发生在属性一级上。其中属性的命名冲突更为常见。处理命名冲突通常也像处理属性冲突一样,应通过讨论、协商等行政手段加以解决。

结构冲突主要包含以下三类冲突:

① 同一对象在不同应用中具有不同的抽象。例如,“职工”在某一局部应用中被当作实体,而在另一局部应用中则被当作属性。其解决方法通常是把属性变换为实体或把实体变换为属性,使同一对象具有相同的抽象。但变换时仍要遵循 7.3.5 小节中讲述的两个准则。

② 同一实体在不同子系统的 E-R 图所包含的属性个数和属性排列次序不完全相同。这是很常见的一类冲突,其原因是不同的局部应用关心的是该实体的不同侧面。解决方法是使该实体的属性取各子系统的 E-R 图中属性的并集,再适当调整属性的次序。

③ 实体间的联系在不同的 E-R 图中为不同的类型。如实体 E1 与 E2 在一个 E-R 图中是多对多联系,在另一个 E-R 图中是一对多联系;又如在一个 E-R 图中 E1 与 E2 发生联系,而在另一个 E-R 图中 E1、E2、E3 三者之间有联系。其解决方法是根据应用的语义对实体联系的类型进行综合或调整。

例如,图 7.28(a)“学生选课管理”子系统中“学生”与“教学班”之间存在多对多的联系“选课”,而图 7.28(b)“教师教学管理”子系统中“学生”“教学班”“教师”之间存在多对多的联系“课程评价”,这两个联系互相不能包含,则在合并两个 E-R 图时就应把它们综合起来,如图 7.28(c)所示。

(2) 消除不必要的冗余,设计基本 E-R 图

在初步 E-R 图中可能存在一些冗余的数据和实体间冗余的联系。所谓冗余的数据是指可由基本数据导出的数据,冗余的联系是指可由其他联系导出的联系。冗余数据和冗余联系容易破坏数据库的完整性,给数据库维护增加困难,应当予以消除。消除了冗余后的初步 E-R 图称为基本 E-R 图。

消除冗余主要采用分析方法,即以数据字典和数据流图为依据,根据数据字典中关于数据项之间逻辑关系的说明来消除冗余。如图 7.29 中,“学生”与“课程”实体之间的“已修”关系(记录学生选修合格及以上的选课信息),可以由“学生”与“教学班”实体之间的“选课”关系以及“教学班”与“课程”实体之间的“开课”关系导出,因此“已修”关系为冗余关系,可以消去。

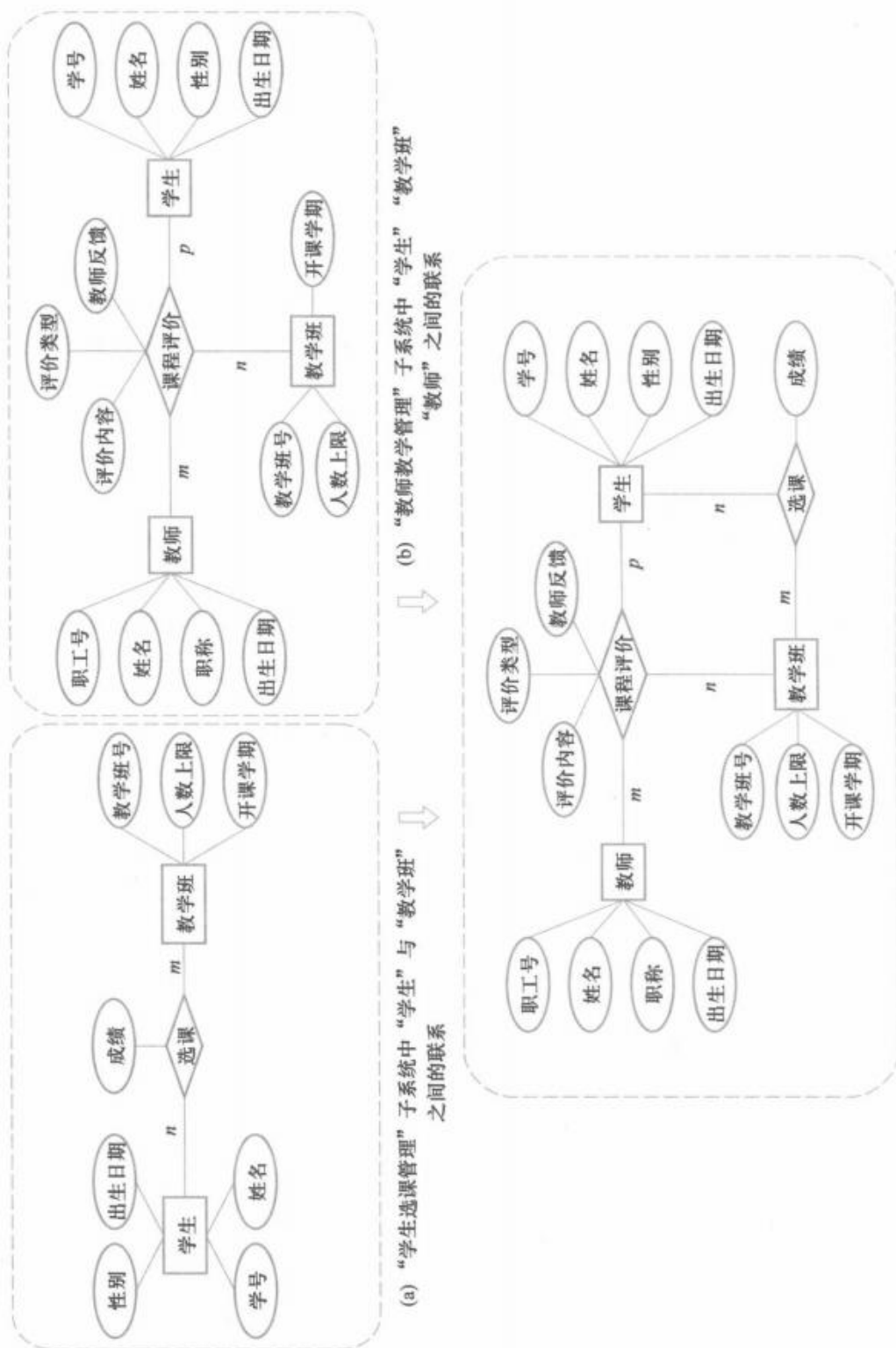


图 7.28 合并两个 E-R 图示例

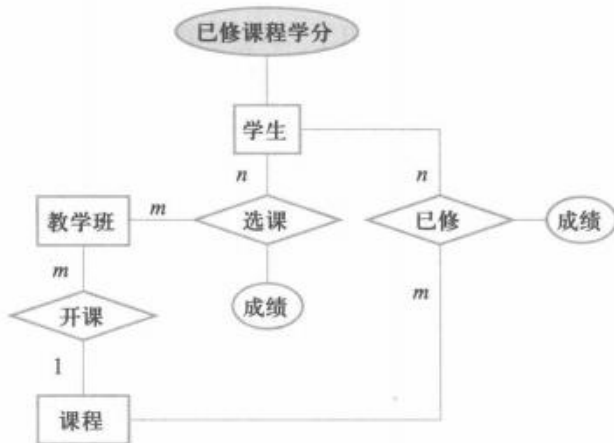


图 7.29 消除冗余示例

但并不是所有的冗余数据与冗余联系都必须加以消除，有时为了提高效率，不得不以冗余信息作为代价。因此在设计数据库概念结构时，哪些冗余信息必须消除，哪些冗余信息允许存在，需要根据用户的整体需求来确定。如果人为地保留了一些冗余数据，则应把数据字典中数据关联的说明作为完整性约束条件。例如，若教务处需要经常查询学生已修课程的学分，如果每次都要查询学生的所有选课信息，统计其中成绩不少于 60 分的课程，再对它们求和，查询效率就太低了。所以应保留学生的冗余属性“已修课程学分”，同时在“已修课程学分”属性值与学生的“选课”成绩之间建立完整性检查，当学生“选课”成绩发生变化时，就触发该完整性检查，对“已修课程学分”值做相应的修改（例如“选课”成绩由原来的默认值或“低于 60 分”修改为“不小于 60 分”时）。

除分析方法外，还可以用规范化理论来消除冗余。在规范化理论中，函数依赖的概念提供了消除冗余联系的形式化工具。具体方法如下：

① 确定分 E-R 图实体之间的数据依赖。实体之间一对一、一对多、多对多的联系可以用实体码之间的函数依赖来表示。例如，学院和系之间一对多的联系可表示为：系编号→学院编号；系和专业之间一对多的联系可表示为：专业编码→系编号；专业和学生之间多对多的联系可表示为：（学号，专业编码）→是否主修，等等。于是有函数依赖集 F_L 。

② 求 F_L 的最小覆盖 G_L ，差集为 $D = F_L - G_L$ 。逐一考察 D 中的函数依赖，确定是否为冗余的联系，若是就把它去掉。由于规范化理论受到泛关系假设的限制，应注意下面两个问题：

- 冗余的联系一定在 D 中，而 D 中的联系不一定是冗余的。
- 当实体之间存在多种联系时，要将实体之间的联系在形式上加以区分。例如教师和系之间一对一的联系“系主任”就要表示为：系主任·职工号→系编号，系编号→系主任·职工号。

[例 7.2] “高校本科教务管理”信息系统的视图集成。

图 7.11、图 7.22(b)、图 7.26 分别给出了“学生学籍管理”“学生选课管理”和“教师教学管理”子系统的分 E-R 图。图 7.30 为“高校本科教务管理”信息系统的基本 E-R 图(这里基本 E-R 图中各实体的属性因篇幅有限从略)。

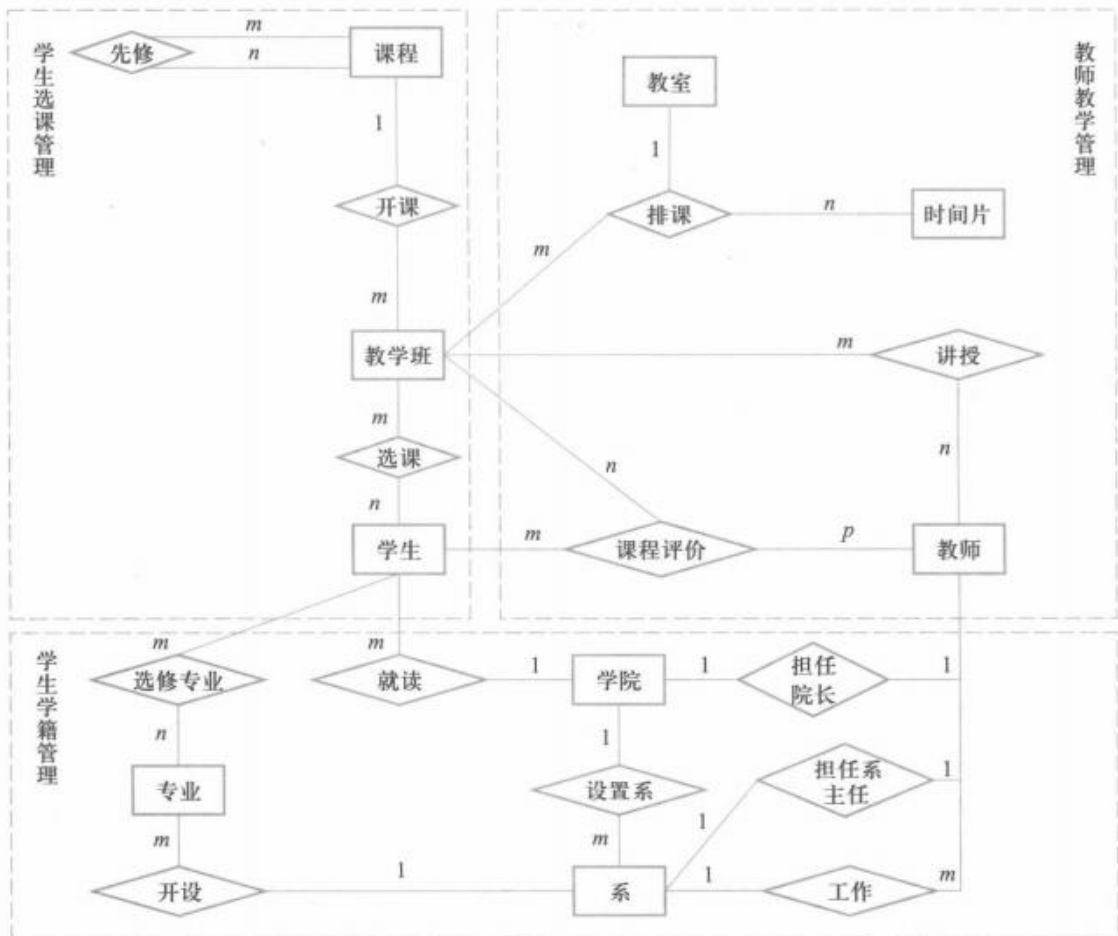


图 7.30 “高校本科教务管理”信息系统的基本 E-R 图

7.4 逻辑结构设计

概念结构是独立于任何一种数据模型的信息结构，逻辑结构设计任务就是把概念结构设计阶段得到的基本 E-R 图转换为与选用数据库管理系统产品所支持的数据模型相符合的逻辑结构。

目前的数据库应用系统普遍采用关系数据库管理系统，所以这里只介绍 E-R 图向关系数据库模型的转换原则与方法。